UNIVERSITÉ ALIOUNE DIOP DE BAMBEY
UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle
Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

Diak-SN

Intelligence Sanitaire & Surveillance Épidémiologique

Santé Publique & Télémédecine

Durée	4 semaines
Équipe	2 étudiants
Domaine	Santé Publique & Télémédecine
Standard	M2 Approche Industrie | New Deal Technologique Sénégal
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Airflow • MLlib • Privacy by Design

1. Contexte et Objectifs

Le Sénégal fait face à une double charge : maladies tropicales (paludisme, dengue, choléra) et montée des maladies chroniques (diabète, HTA) dans un contexte de désert médical dans les régions. Ce projet construit un système de surveillance sanitaire temps réel qui prédit les pics de consultations par district et optimise la distribution des médicaments essentiels.

> ℹ️ **New Deal Technologique Sénégal**
> Ce projet s'inscrit dans la Stratégie Sénégal Numérique 2025-2035 et le Plan Sénégal Émergent. L'objectif est de former des ingénieurs Big Data capables de construire des solutions Data-Driven directement applicables aux défis de développement du Sénégal.

1.1 Objectifs Techniques

#	Composant	Description
1	**Ingestion multi-sources**	rapports DHIS2, météo ANACIM, données démographiques ANSD
2	**Prédiction de pics de consultation par district et type de pathologie**	Prédiction de pics de consultation par district et type de pathologie
3	**Détection précoce d'épidémies (seuil de Farrington adapté aux données sénégalaises)**	Détection précoce d'épidémies (seuil de Farrington adapté aux données sénégalaises)
4	**Privacy by Design**	anonymisation des dossiers patients (conformité loi 2008-12)
5	**Dashboard de recommandations**	ruptures de stock médicaments, transferts de patients

1.2 Architecture End-to-End

📄 Architecture

```
[ SOURCES ]  ──▶  [ NiFi ]  ──▶  [ KAFKA ]  ──▶  [ SPARK ML ]  ──▶  [ STORAGE ]
 DHIS2 API      Orchestration    Topic:           Prédiction          HBase/HDFS
 Météo ANACIM   & Routage        diak_cas         Pic consult.        Gold Layer
 ANSD Démo                       diak_stocks      Alerte épidémie          │
                                                          │           [ ANALYTICS ]
                                                          ▼             Apache Hive
                                              [ ALERT LAYER ]       vue_districts
                                              Épidémie détectée     vue_stocks
```

2. Prérequis & Infrastructure

ℹ️ Configuration minimale

RAM : 16 Go. CPU : 4 cœurs. OS : Ubuntu 22.04 LTS. Docker Engine 24.x. Java 11 requis par Spark.

setup.sh

```
# Démarrage infrastructure
mkdir -p data dags models nifi_templates
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Démarrer le simulateur
python kafka_producer_diak_sn.py &

# Interfaces web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082
```

requirements.txt

```
# requirements.txt — Diak-SN
pyspark==3.3.2
kafka-python==2.0.2
pandera==0.17.2
pandas==2.0.3
apache-airflow==2.7.3
mlflow==2.8.1
pyhive==0.7.0
happybase==1.2.0
matplotlib==3.7.3
seaborn==0.12.2
faker==30.1.0
scipy==1.11.4
numpy==1.24.4
```

3. Codes Sources Complets & Fonctionnels

3.1 Initialisation HBase — hbase_setup.py

> ⚠️ **Fichier obligatoire**
> Crée les 3 tables HBase de Diak-SN. À exécuter avant le démarrage du pipeline.

📄 **hbase_setup.py**

```
# hbase_setup.py — Tables HBase Diak-SN
import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_diak_sn_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        b'diak:cas_temps_reel': {'b'meta': {'max_versions': 1}},  # Cas par pathologie
(TTL 72h)
        b'diak:alertes_epidemie': {'b'meta': {'max_versions': 1}},  # Alertes district +
pathologie
        b'diak:stocks_medicaments': {'b'meta': {'max_versions': 1}},  # Niveaux de stock
par district
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée ✓')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_diak_sn_tables()
```

3.2 Simulateur de Données — kafka_producer

> ℹ️ **Données simulées réalistes**
> Le simulateur génère des données calées sur les réalités de Santé Publique & Télémédecine au Sénégal. Les champs PII (noms, IDs) sont présents dans le simulateur et seront anonymisés par la Privacy Layer Spark.

📄 **kafka_producer_diak_sn.py**

```
# kafka_producer_diak.py — Simulateur cas médicaux + stocks
from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime
import numpy as np

random.seed(42); np.random.seed(42)
DISTRICTS = ['DAKAR','THIES','KAOLACK','ZIGUINCHOR','SAINT_LOUIS','LOUGA','TAMBACOUNDA']
PATHOLOGIES = ['PALUDISME','DENGUE','CHOLERA','HTA','DIABETE','DIARRHEE','IRA']
# Saisonnalité : paludisme pic hivernage (juil-oct), choléra saison sèche
PROB_SAISON = {
    'PALUDISME': lambda m: 0.4 if 7<=m<=10 else 0.1,
    'DENGUE':    lambda m: 0.3 if 8<=m<=11 else 0.05,
    'CHOLERA':   lambda m: 0.2 if m in [1,2,3,4,5] else 0.05,
    'HTA':       lambda m: 0.15, 'DIABETE': lambda m: 0.12,
    'DIARRHEE':  lambda m: 0.2, 'IRA': lambda m: 0.18,
```

```
}
STOCK_BASE =
{'COARTEM':500,'PARACETAMOL':2000,'SRO':800,'METFORMINE':300,'AMLODIPINE':400}

producer = KafkaProducer(bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v, ensure_ascii=False).encode())

def gen_cas():
    district  = random.choice(DISTRICTS)
    m         = datetime.utcnow().month
    pathologie= random.choices(PATHOLOGIES,
                    weights=[PROB_SAISON[p](m) for p in PATHOLOGIES])[0]
    return {
        'cas_id':       str(uuid.uuid4()),
        'patient_id':   f'PAT_{uuid.uuid4().hex[:10].upper()}',  # Sera anonymisé
        'district':     district,
        'pathologie':   pathologie,
        'age_tranche':  random.choice(['0-5','6-14','15-49','50+']),
        'sexe':         random.choice(['M','F']),
        'gravite':      random.choice(['LEGER','MODERE','GRAVE']),
        'timestamp':    datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

def gen_stock():
    district  = random.choice(DISTRICTS)
    medicament= random.choice(list(STOCK_BASE.keys()))
    base      = STOCK_BASE[medicament]
    return {
        'district':     district,
        'medicament':   medicament,
        'stock_actuel': max(0, round(base * random.uniform(0.1, 1.5))),
        'stock_mini':   round(base * 0.2),
        'timestamp':    datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

if __name__ == '__main__':
    print('Simulateur Diak-SN démarré...')
    while True:
        for _ in range(5): producer.send('diak_cas_raw', gen_cas())
        producer.send('diak_stocks_raw', gen_stock())
        producer.flush(); time.sleep(3)
```

3.3 Pipeline Spark Streaming — streaming

> ⚠️ **Privacy critique**
> Drop des champs PII après anonymisation SHA-256. Les données sensibles ne doivent jamais apparaître dans HBase ou Hive.

streaming_diak_sn.py

```
# streaming_diak.py — Pipeline Spark Streaming Diak-SN
# Détection épidémie + seuil adaptatif + alerte rupture stock
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, count, avg, stddev, when, coalesce, current_timestamp
)
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

SALT    = os.environ.get('DIAK_SECRET_SALT','default_salt')
BROKERS = 'kafka:9092'
spark   = SparkSession.builder.appName('Diak_SN_Streaming') \
    .config('spark.sql.shuffle.partitions','4').getOrCreate()
spark.sparkContext.setLogLevel('WARN')

cas_schema = StructType([
    StructField('cas_id',     StringType(), True),
    StructField('patient_id', StringType(), True),
    StructField('district',   StringType(), True),
    StructField('pathologie', StringType(), True),
    StructField('age_tranche',StringType(), True),
    StructField('sexe',       StringType(), True),
    StructField('gravite',    StringType(), True),
    StructField('timestamp',  StringType(), True),
])

cas_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('subscribe','diak_cas_raw').load()
    .select(from_json(col('value').cast('string'), cas_schema).alias('d')).select('d.*')
    .withColumn('event_ts', current_timestamp())
    # Privacy — drop patient_id, conserver uniquement cas_id_secure
    .withColumn('cas_id_secure', sha2(concat(col('patient_id'),lit(SALT)),256))
    .drop('patient_id')
)

# Détection épidémie : fenêtre 7 jours, watermark 1 jour
epidemie_df = (cas_df
    .withWatermark('event_ts','1 day')
    .groupBy(window('event_ts','7 days','1 day'), 'district','pathologie')
    .agg(count('cas_id').alias('nb_cas_7j'))
    .withColumn('alerte_epidemie',
        # Seuil simplifié : > 50 cas/7j = ALERTE, > 100 = CRITIQUE
        when(col('nb_cas_7j') > 100, lit('CRITIQUE'))
        .when(col('nb_cas_7j') > 50,  lit('ALERTE'))
        .otherwise(lit('NORMAL')))
)

q1 = (epidemie_df.select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('topic','diak_alerts')
    .option('checkpointLocation','/tmp/diak_epidemie_ckpt')
    .outputMode('update').start())
q1.awaitTermination()
```

3.4 Tables Hive & Vues Analytics — hive_setup.sql

Les vues Hive utilisent systématiquement COALESCE sur les colonnes nullables pour éviter les NULL silencieux dans les agrégations.

hive_setup.sql

```
-- hive_setup.sql — Tables Hive Diak-SN
CREATE DATABASE IF NOT EXISTS diak_sn COMMENT 'Santé Sénégal — UADB 2025';
USE diak_sn;

CREATE TABLE IF NOT EXISTS cas_journaliers (
    cas_id_secure STRING, district STRING, pathologie STRING,
    age_tranche STRING, sexe STRING, gravite STRING, ingestion_ts TIMESTAMP
) PARTITIONED BY (date_obs STRING, pathologie_part STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

CREATE TABLE IF NOT EXISTS stocks_medicaments (
    district STRING, medicament STRING,
    stock_actuel INT, stock_mini INT, ingestion_ts TIMESTAMP
) STORED AS ORC;

-- Vue : Alertes épidémie par district
CREATE OR REPLACE VIEW vue_alertes_districts AS
SELECT district, pathologie,
    COUNT(*) AS nb_cas_7j,
    CASE WHEN COUNT(*) > 100 THEN 'CRITIQUE'
         WHEN COUNT(*) > 50  THEN 'ALERTE'
         ELSE 'NORMAL' END AS statut,
    MIN(ingestion_ts) AS premier_cas
FROM cas_journaliers
WHERE date_obs >= DATE_SUB(CURRENT_DATE(),7)
GROUP BY district, pathologie
ORDER BY nb_cas_7j DESC;

-- Vue : Ruptures de stock imminentes
CREATE OR REPLACE VIEW vue_ruptures_stock AS
SELECT district, medicament, stock_actuel, stock_mini,
    ROUND((stock_actuel / COALESCE(stock_mini,1)) * 100, 1) AS pct_stock
FROM stocks_medicaments
WHERE stock_actuel <= stock_mini * 1.2  -- Seuil : 20% au-dessus du minimum
ORDER BY pct_stock ASC;
```

3.5 DAG Airflow MLOps

Le BranchPythonOperator détecte automatiquement les dérives et déclenche le réentraînement du modèle si nécessaire. Les recommandations sont écrites dans HBase pour un accès temps réel.

dags/diak_sn_dag.py

```
# dags/diak_sn_dag.py — DAG Airflow Diak-SN
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('diak_dag')
default_args = {'owner':'seye_ahmed','retries':2,'retry_delay':timedelta(minutes=5)}

def check_epidemie(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cur  = conn.cursor()
```

```
    cur.execute("SELECT COUNT(*) FROM diak_sn.vue_alertes_districts WHERE
statut='CRITIQUE'")
    n = cur.fetchone()[0]
    logger.info(f'Districts en statut CRITIQUE : {n}')
    return 'retrain_model' if n > 1 else 'generate_recommendations'

def retrain_model(**ctx):
    subprocess.run(['spark-submit','--master','spark://spark-master:7077',
        '/opt/models/train_pic_consult.py','--mode','train'],check=True,timeout=3600)

def generate_recommendations(**ctx):
    from pyhive import hive
    import happybase
    conn_hbase = happybase.Connection('hbase',port=9090); conn_hbase.open()
    conn_hive  = hive.Connection(host='hive-metastore',port=10000)
    cur = conn_hive.cursor()
    cur.execute('SELECT district,pathologie,nb_cas_7j,statut FROM
diak_sn.vue_alertes_districts WHERE statut != "NORMAL"')
    table = conn_hbase.table(b'diak:alertes_epidemie')
    from datetime import datetime
    for row in cur.fetchall():
        dist,path,nb,statut = row
        table.put(f'{dist}_{path}'.encode(), {
            b'alerte:district':  dist.encode(),
            b'alerte:pathologie':path.encode(),
            b'alerte:nb_cas':    str(nb).encode(),
            b'alerte:statut':    statut.encode(),
            b'alerte:ts':        datetime.utcnow().isoformat().encode(),
        })
    conn_hbase.close()

with DAG('diak_sn_monitoring', default_args=default_args,
         schedule_interval='0 6 * * *', start_date=days_ago(1), catchup=False,
         tags=['diak-sn','sante','epidemie']) as dag:
    start  = DummyOperator(task_id='start')
    end    = DummyOperator(task_id='end')
    branch = BranchPythonOperator(task_id='check_epidemie',
                                  python_callable=check_epidemie,provide_context=True)
    t_train= PythonOperator(task_id='retrain_model',
                            python_callable=retrain_model,provide_context=True)
    t_reco = PythonOperator(task_id='generate_recommendations',

python_callable=generate_recommendations,provide_context=True)
    start >> branch >> [t_train, t_reco]
    t_train >> t_reco >> end; t_reco >> end
```

4. Livrables Attendus

#	Livrable	Contenu attendu
1	**Pipeline de Données**	Code source complet (GitHub) + template NiFi. Données visibles dans HDFS, topic Kafka diak_cas_raw actif.
2	**Modèle ML & MLOps**	Script d'entraînement + RMSE/R^2 documentés + DAG Airflow avec BranchPythonOperator fonctionnel.
3	**Catalogue Hive**	Tables créées, vues fonctionnelles avec au moins 1 alerte visible dans HBase en démonstration.
4	**Dashboard & Recommandations**	Dashboard graphique (Matplotlib) + recommandations concrètes contextualisées au Santé Publique & Télémédecine sénégalais.

5. Barème de Notation — /20 points

> ℹ️ **Barème différencié 4 niveaux**
> Chaque critère est évalué selon 4 niveaux de maîtrise. La démonstration live en soutenance est obligatoire. Bonus documentation +2 pts.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi Multi-Sources	3 pts	NiFi absent	1 source/3	2 sources + topics	DHIS2+météo+ANSD, 2 topics distincts, messages conformes
Validation & Privacy (Loi 2008-12)	4 pts	Aucune validation, patient_id en clair	Schéma partiel	Schéma + SHA-256 sans suppression patient_id	Pandera + drop(patient_id, nom_patient) + 0 PII dans HBase
Streaming & Détection Épidémie	6 pts	Pas de Spark	Lecture seule	Fenêtres 7j + seuil fixe	Seuil adaptatif (AVG+2*STDDEV 30j) + watermarks + COALESCE
Prédiction Pics & MLOps	4 pts	Pas de modèle	Règles expertes seules	RF entraîné, pas de DAG	RF + DAG conditionnel + RMSE/R^2 documentés + MLflow
Stocks & Recommandations	3 pts	Pas de stock	Tables vides	Vues Hive stocks OK	Alerte rupture stock + recommandation transfert inter-district
TOTAL	20 pts				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec docker compose up + captures d'écran	+0.5 pt
Commentaires de code en français + docstrings	+0.5 pt
Tests pytest sur les schémas Pandera et les UDFs	+0.5 pt
Démo live d'une alerte HBase sans erreur en soutenance	+0.5 pt

🏥 **Bon courage pour Diak-SN !**

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel